

大语言模型有效微调实验课

刘云

2024 年 12 月 4 日

1 背景简述

1.1 常用基线模型介绍

1.1.1 BERT（双向编码器表示的变换器）

类型: 仅编码器模型。

开发者: Google AI。

描述: BERT 通过同时关注上下文信息，生成深度的双向语义表示，特别适合自然语言理解任务。

预训练任务: 掩码语言模型（MLM）：随机屏蔽输入中的部分词，并让模型预测这些词。下一句预测（NSP）：判断两段句子是否连续。

应用场景: 文本分类、问答系统、实体识别等。

特点: 强调语义理解，擅长上下文敏感的语言任务。

1.1.2 BART（双向和自回归变换器）

类型: 编码器-解码器模型。

开发者: Facebook AI (Meta AI)。

描述: BART 结合了 BERT 的双向编码能力和 GPT 的自回归生成能力，是一个通用的序列到序列模型。预训练任务: 作为去噪自动编码器，学习从被破坏（如屏蔽、打乱）的输入文本重建原始文本。

应用场景: 文本摘要、翻译、对话生成等。

特点: 既擅长理解任务，又擅长生成任务，尤其适用于序列到序列的转换。

1.1.3 GPT (生成式预训练变换器)

类型: 仅解码器模型, 自回归生成。

开发者: OpenAI。

描述: GPT 专注于生成任务, 基于已有上下文生成后续内容, 擅长生成连贯、语义合理的文本。预训练任务: 因果语言模型 (Causal Language Modeling), 预测序列中的下一个词。应用场景: 文本生成、对话系统、创意写作等。

特点: 在生成任务上表现卓越, 但对上下文理解的能力不如 BERT。

1.1.4 T5 (文本到文本转换器)

类型: 编码器-解码器模型。

开发者: Google AI。

描述: T5 将所有自然语言处理任务统一为“文本到文本”的形式, 输入和输出均为文本序列。这种统一框架使模型可以灵活地适应多种任务。

预训练任务: 通过去噪重构输入文本完成预训练 (类似于 BART)。

应用场景: 翻译、摘要、问答、多任务学习等。

特点: 任务通用性强, 适合多任务协同训练。

1.2 高效微调方法简述

1.2.1 增量式微调 (Addition-based) -前缀 (Prefix) 微调

前缀微调 (Prefix Tuning) 属于 Addition-based 方法, 将一系列特定向量附加到输入或者 prefix 的开头。只有前缀参数被优化并添加到模型的每一层的隐藏状态中。前缀微调在资源有限、任务多样化的场景下具有显著的优势。前缀优化存储的参数比 full fine-tuning 的模型少 1000 倍。

在输入 token 之前构造一段与任务相关的虚拟令牌作为前缀 (Prefix), 然后训练的时候只更新前缀的参数, 而预训练模型中的其他参数固定不变。

针对不同的模型结构, 前缀微调需要构建不同的前缀:

(1) 回归架构模型: 在输入之前添加前缀, 得到 $z = [\text{PREFIX}; x; y]$, 合适的上文能够在固定预训练模型的情况下引导生成下文, 如 GPT-3 的上下文学习。

(2) 编码器-解码器架构模型: 编码器和解码器都需要增加前缀, 得到 $z = [\text{PREFIX}; x; \text{PREFIX0}; y]$ 。编码器端增加前缀用来引导输入部分的编码, 解码器端增加前缀用来引导后续 token 的生成。

1.2.2 重参数化微调 (Reparametrization-based) - LoRA 微调

LoRA 微调指通过在预训练模型中引入低秩结构来实现高效的参数微调。其核心思想是通过低秩分解来修改模型的权重矩阵，使其分解为较低维度的因子，从而减少在微调过程中需要更新的参数数量。

2 大模型常用 API 接口

2.1 环境搭建

本实验可以采用 Anaconda 或者 venv 两种方式部署虚拟环境：

1.Anaconda 创建虚拟环境

```
# 创建虚拟环境
conda create -n llm_peft python = 3.7
# 激活虚拟环境
conda activate llm_peft
# 安装必要的包
conda install openai
```

2.venv 创建虚拟环境

venv 模块是 Python 自带的库，专门用于创建虚拟环境。运行以下命令创建虚拟环境：

```
# 打开命令行工具（终端、命令提示符等），导航到你想要创建虚拟环境的目录。
# 创建虚拟环境
python3 -m venv llm_peft
# 激活虚拟环境 (windows cmd, 非 powershell)
call llm_peft/Scripts/activate.bat
# 安装必要的包
pip install openai
```

3. 微调包安装

由于 LORA,AdaLORA 都集成在 PEFT 上了，所以在使用的時候需要安装 PEFT 包：

- (1) PyPI 安装：pip install peft
- (2) 源码安装：

```
pip install git+https://github.com/huggingface/peft
或者
git clone https://github.com/huggingface/peft
cd peft
pip install -e .
```

3 实验

3.1 中英文翻译: T5+ 前缀微调

题目: 使用前缀微调方法微调大型语言模型以执行中文翻译成英文的任务。假设你已经加载了预训练的 T5 模型, 现在想通过前缀微调方法使其适应中文翻译成英文的任务。

原文本: “这是一个关于大模型微调的问题, 希望通过前缀微调方法来适应中文翻译成英文的任务。”

备注: T5 属于 Encoder-Decoder 架构, Encoder (编码器): 负责处理输入序列, 将其转换为上下文感知的表示 (即隐藏状态)。Decoder (解码器): 在获取输入的表示后, 生成目标序列的输出。针对编码器-解码器架构: Encoder 和 Decoder 都增加了前缀, 得到 $z = [\text{PREFIX}; x; \text{PREFIX0}; y]$ 。Encoder 端增加前缀是为了引导输入部分的编码, Decoder 端增加前缀是为了引导后续 token 的生成。模型训练步骤如下所示:

(1) 加载 T5 模型和 Tokenizer。我们使用 “t5-base”

```
model_name = "D:/data/PEFT/t5-base"
tokenizer = T5Tokenizer.from_pretrained(model_name)
model = T5ForConditionalGeneration.from_pretrained(model_name)
```

T5 将所有任务 (如翻译、摘要、文本生成等) 视为文本到文本的转换任务。T5ForConditionalGeneration: 专门针对 T5 模型设计的类, 用于条件生成任务, 输入是一个序列, 模型根据输入序列生成一个相应的输出序列, 具备专门的优化和预训练权重。

(2) 调用 PrefixTuningConfig, 配置前缀微调

```
prefix_tuning_config = PrefixTuningConfig(
```

```
task_type="SEQ_2_SEQ_LM", # 指定任务类型, 序列到序列任务
num_layers=4, # 选择微调模型的层数
num_virtual_tokens=8, # 虚拟 token 的数量, 换句话说就是提示 (prompt)
prefix_projection=True, # 是否投影前缀嵌入 (token), 默认值为 False, 表示使用 P-Tuning v2, 如果为 true, 则表示使用 Prefix Tuning。
)# inference_mode: 是否在推理模式下使用 Peft 模型。
```

(3) 获取微调模型

```
peft_model = get_peft_model(model, prefix_tuning_config)
Debug 参考: https://www.cnblogs.com/tuyuge/p/17612914.html, 详细分析了调用步骤。
```

(4) 加载数据集

从 hugging face 上下载中-英文数据集, 这里采用 “en-zh-dataset”, 将此数据集下载到本地, 调用 load_dataset() 将路径下的数据集处理成 “arrow” 文件, 后续可以直接调用 “arrow” 数据。

```
dataset = load_dataset('D:/data/PEFT/en-zh-dataset', cache_dir='D:/data/PEFT/en-zh-dataset/cache')
arrow_dir='D:/data/PEFT/en-zh-dataset/arrow/'
# dataset.save_to_disk(arrow_dir)
dataset=load_from_disk(arrow_dir)
```

(5) 根据数据集格式处理数据集 (preprocess_function), 获取训练集和测试集

(6) 设置优化器:

```
optimizer = AdamW(peft_model.parameters(), lr=1e-5)
```

(7) 训练模型: peft_model.train()

(8) 模型评估和测试

(9) 模型训练完成, 保存高效微调部分的模型权重以供模型推理

3.2 文本摘要：BERT+LoRA

题目：使用 LoRA 微调大型语言模型以执行文本摘要任务。假设你已经加载了预训练的 BERT 模型，现在想通过 LoRA 微调使其适应文本摘要任务。

原文本：“自然语言处理（Natural Language Processing, NLP）是 AI 领域中与计算机和人类语言之间交互的研究。NLP 的目标是使计算机能够理解、解释、生成人类语言，使计算机与人的交互更加自然。它涉及文本处理、语音处理、机器翻译等多个方面的任务。”使用 LoRA 微调方法，将上述文本进行摘要生成，提取出主要信息。

备注：BART(Bidirectional and Auto-Regressive Transformers)属于 Encoder-Decoder 架构 BART 结合了自回归和双向的特性，但在架构上，它仍然是一个 Encoder-Decoder 模型。Encoder（编码器）：BART 的编码器是一个类似于 BERT 的双向 Transformer 编码器，对输入文本的双向表示进行编码，捕获上下文信息。Decoder（解码器）：BART 的解码器是自回归的，即在生成每个新的词时，它会依赖之前生成的词，这类似于 GPT 的解码器结构。模型训练步骤如下所示：

(1) 加载 BART 模型和 Tokenizer。我们使用使用“facebook/bart-large-cnn”模型，它是专门为文本摘要任务优化的 BART 模型。BartTokenizer 用于将文本转换为模型能够理解的 tokens。

```
model_name = "D:/data/PEFT/facebook/bart-large-cnn"
model = BartForConditionalGeneration.from_pretrained(model_name)
tokenizer = BartTokenizer.from_pretrained(model_name)
```

(2) 调用 LoraConfig，配置 LoRA 微调 lora_config = LoraConfig(
r=8, # 低秩矩阵的秩
lora_alpha=16, # 权重衰减系数
lora_dropout=0.1, # dropout 率
bias="none", # 是否使用偏置
task_type="SEQ_2_SEQ_LM" # 任务类型：序列到序列
)

(3) 获取微调模型

```
peft_model = get_peft_model(model, lora_config)
```

(4) 加载数据集

使用”cnn_dailymail”数据集作为摘要任务的训练数据集，同样的将此数据集下载到本地，利用 load_dataset() 将路径下的数据集处理成 “arrow” 文件，以便后续可以直接调用 “arrow” 数据。

```
# dataset = load_dataset("D:/data/PEFT/cnn_dailymail", "3.0.0")
arrow_dir='D:/data/PEFT/cnn_dailymail/3.0.0/arrow/'
dataset.save_to_disk(arrow_dir)
dataset=load_from_disk(arrow_dir)
```

(5) 根据数据集格式处理数据集 (preprocess_function), 获取训练集和测试集

(6) 设置优化器:

```
optimizer = AdamW(peft_model.parameters(), lr=1e-5)
```

(7) 训练模型: peft_model.train()

(8) 模型评估和测试

(9) 模型训练完成，保存高效微调部分的模型权重以供模型推理